

Building and Securing a REST API

1. Introduction

The MoMo SMS API provides read and write access to a mobile money transaction log. The data behind it is a corpus of 1,691 SMS messages extracted from an MTN Mobile Money account, parsed from `modified_sms_v2.xml` and normalized into structured records.

The API is written in plain Python using the `http.server` from the standard library. There are no external dependencies, no web framework, and no database. All records live in memory while the server is running, and every endpoint is protected by HTTP Basic Authentication.

Why API security matters

An API is the way every client application gets to the data behind it, which means it is also the way anyone else would get to that data if it were left open. For a mobile money system, the data behind that door is about as sensitive as it gets, it says how much money moved, who sent it, who received it, their phone numbers, and what the balance was afterwards.

Someone who can reach an unprotected version of this API can read a customer's entire financial history. If the write endpoints are open too, they can change or delete it.

When people talk about securing an API they usually mean three different things, and it helps to keep them apart:

- **Authentication** is working out who is making the request.
- **Authorization** is deciding whether that person is allowed to do the thing they are asking for.
- **Confidentiality** is stopping anyone else from reading the exchange while it crosses the network. That is what HTTPS gives you.

You need all three. Checking a password carefully does not help if the password is sent in a form anyone can read. Encrypting the connection does not help if every user who logs in is allowed to delete everything.

This project covers the first of the three, using HTTP Basic Authentication, because that is what the assignment asked for. Every endpoint turns away unauthenticated requests with a 401. Section 7 goes through where that approach falls short.

What this document covers

This report is written for two readers: someone who needs to integrate with the API, and someone who wants to know how it was built. It covers authentication, the shape of a transaction, every endpoint with request and response examples, the error codes you will see, the search-performance comparison that shaped the design, and the security properties you should be aware of.

- **Base URL:** `http://localhost:8000`
- **Content type:** `application/json` for both requests and responses
- **Authentication:** required on every endpoint

2. Authentication

Every request must carry an `Authorization` header containing your credentials. The header uses the standard Basic Authentication format: the username and password joined with a colon and encoded as base64.

Authorization: Basic YWRtaW46bW9tbzIwMjU=

Using curl, the `-u` flag handles this for you:

```
curl -u admin:momo2025 http://localhost:8000/transactions
```

The server is configured with the username `admin` and password `momo2025` by default. Both can be overridden at startup by setting the `API_USERNAME` and `API_PASSWORD` environment variables, which is what a real deployment would do rather than using the shipped defaults.

2.1 Handling authentication failures

If the header is missing, malformed, or the credentials are wrong, the server responds with a 401 status and the `WWW-Authenticate` header:

```
HTTP/1.0 401 Unauthorized
```

```
WWW-Authenticate: Basic realm="MoMo Transactions API"
Content-Type: application/json
```

```
{
  "error": "Unauthorized: invalid username or password",
  "status": 401
}
```

Clients should treat any 401 as "your credentials are wrong or missing" and either prompt the user or abort the request. There is no lockout and no rate limiting, so retrying with correct credentials after a 401 is fine.

3. The Transaction Object

Every transaction in the system has the same seventeen fields. Fields that the original SMS did not mention are stored as `null`. It is normal for a given record to have several nulls.

```
json
{
  "id": 1,
  "transaction_type": "incoming_money",
  "amount": 2000.0,
  "fee": null,
  "new_balance": 2000.0,
  "sender": "Jane Smith",
  "sender_phone": "*****013",
  "receiver": null,
  "receiver_phone": null,
  "receiver_code": null,
  "account": null,
  "financial_transaction_id": "76662021700",
  "timestamp": "2024-05-10T16:30:51",
  "readable_date": "10 May 2024 4:30:58 PM",
  "service_center": "+250788110381",
  "address": "M-Money",
  "body": "You have received 2000 RWF from Jane Smith (*****013)
on your mobile money account at 2024-05-10 16:30:51. Message from
```

sender: . Your new balance:2000 RWF. Financial Transaction Id:
76662021700."

}

3.1 Field reference

Field	Type	Description
id	integer	The record's id. The server assigns it.
transaction_type	string	What kind of transaction is this? See §3.2.
amount	number or null	How much money moved, in RWF.
fee	number or null	The fee charged, if the SMS mentions it.
new_balance	number or null	The balance after the transaction.
Sender	string or null	Who sent the money, when it is incoming.
sender_phone	string or null	The sender's phone, sometimes masked.
receiver	string or null	Who received the money, when outgoing.
receiver_phone	string or null	The receiver's phone number.
receiver_code	string or null	The merchant code, for code-holder payments.
account	string or null	The mobile money account number.
financial_transaction_id	string or null	MTN's own reference for the transaction.
timestamp	string or null	When it happened, in ISO-8601.
readable_date	string or null	The date as it appeared in the original SMS.
service_center	string or null	The SMS service centre number.
address	string or null	Almost always M-Money.
body	string or null	The raw SMS text.

3.2 Transaction types

Every record falls into one of twelve categories, decided by the parser from the wording of the SMS:

Type	Records
------	---------

payment_to_code_holder	658
transfer_to_mobile	585
bank_deposit	249
incoming_money	63
bill_or_airtime_payment	53
third_party_payment	36
bundle_purchase_notice	21
otp	8
bank_transfer	6
failed_transaction	5
withdrawal	3
reversal / merchant_payment	2 each

4. Endpoints

The API exposes five endpoints under the `/transactions` path.

4.1 List all transactions — `GET /transactions`

Returns every transaction in the store, sorted by id.

Request:

```
curl -u admin:momo2025 http://localhost:8000/transactions
```

Response — 200 OK:

json

```
{
  "count": 1691,
  "transactions": [
    {
      "id": 1,
      "transaction_type": "incoming_money",
      "amount": 2000.0,
      "sender": "Jane Smith",
      "timestamp": "2024-05-10T16:30:51",
      "body": "You have received 2000 RWF from Jane Smith ..."
    }
  ]
}
```

Pagination. Add `?page=N&limit=M` to fetch a slice.

```
curl -u admin:momo2025
"http://localhost:8000/transactions?page=1&limit=10"
```

Search by type. Add `/search?type=SEND` to filter by transaction type.

```
curl -u admin:momo2025
"http://localhost:8000/transactions/search?type=SEND"
```

Errors:

Code	Meaning
401	Credentials missing or wrong

4.2 Get one transaction – GET /transactions/{id}

Returns a single record by id. The server uses an in-memory dictionary keyed by id, so this lookup runs in constant time regardless of how many records are in the store.

Request:

```
curl -u admin:momo2025 http://localhost:8000/transactions/1
```

Response – 200 OK:

json

```
{
  "success": true,
  "data": {
    "id": 1,
    "transaction_type": "incoming_money",
    "amount": 2000.0,
    "new_balance": 2000.0,
    "sender": "Jane Smith",
    "sender_phone": "*****013",
    "financial_transaction_id": "76662021700",
    "timestamp": "2024-05-10T16:30:51"
  }
}
```

Response — 404 Not Found (if the id does not exist):

json

```
{
  "error": true,
  "status": 404,
  "message": "Transaction 99999 not found"
}
```

Errors:

Code	Meaning
400	The id in the URL was not a number
401	Credentials missing or wrong
404	No record with that id

4.3 Create a transaction — POST /transactions

Adds a new transaction to the store. The server assigns the id; do not send one.

Required fields: `transaction_type`, `amount`. All others are optional and default to `null`.

Request:

```
curl -u admin:momo2025 -X POST http://localhost:8000/transactions \
-H "Content-Type: application/json" \
-d '{
  "transaction_type": "incoming_money",
  "amount": 7500,
  "fee": 0,
  "sender": "Chol Deng",
  "receiver": "Test Account",
  "timestamp": "2025-09-19T10:00:00"
}'
```

Response — 201 Created:

```
json
{
  "success": true,
  "message": "Transaction created",
  "data": {
    "id": 1692,
    "transaction_type": "incoming_money",
    "amount": 7500.0,
    "fee": 0.0,
    "sender": "Chol Deng",
    "receiver": "Test Account",
    "timestamp": "2025-09-19T10:00:00"
  }
}
```

The id the server picked is one past the highest existing id.

Response — 400 Bad Request:

```
json
{
  "error": "Missing required field(s): transaction_type",
  "status": 400
}
```

Errors:

Code	Meaning
400	Body was not JSON, required field missing, unknown field present, or a number was invalid
401	Credentials missing or wrong

4.4 Update a transaction — PUT /transactions/{id}

Updates an existing record. Send only the fields you want to change; anything you leave out keeps its current value.

Request:

```
curl -u admin:momo2025 -X PUT
http://localhost:8000/transactions/1692 \
  -H "Content-Type: application/json" \
  -d '{"amount": 9900, "receiver": "Updated Account"}'
```

Response — 200 OK:

```
json
{
  "success": true,
  "message": "Transaction updated",
  "data": {
    "id": 1692,
    "transaction_type": "incoming_money",
    "amount": 9900.0,
    "fee": 0.0,
    "sender": "Chol Deng",
    "receiver": "Updated Account",
    "timestamp": "2025-09-19T10:00:00"
  }
}
```

The response body is the record as it looks after the update, so no second GET is needed.

Errors:

Code	Meaning
400	Bad JSON, unknown field, invalid number, or no valid fields to update
401	Credentials missing or wrong
404	No record with that id

4.5 Delete a transaction — **DELETE /transactions/{id}**

Removes a record permanently. The response returns the deleted record so the client can confirm what was removed.

Request:

```
curl -u admin:momo2025 -X DELETE
http://localhost:8000/transactions/1692
```

Response — 200 OK:

```
json
{
  "message": "Transaction 1692 deleted",
  "deleted": {
    "id": 1692,
    "transaction_type": "incoming_money",
    "amount": 9900.0,
    "receiver": "Updated Account"
  }
}
```

Errors:

Code	Meaning
401	Credentials missing or wrong
404	No record with that id

5. Error Codes

Every error in the API returns the same shape, so clients can write one handler for all of them:

json

```
{
  "error": "<description>",
  "status": <http status code>
}
```

Code	Name	When
200	OK	GET, PUT, or DELETE succeeded
201	Created	POST succeeded
400	Bad Request	Broken JSON, missing required field, unknown field, invalid number
401	Unauthorized	Missing, malformed, or wrong credentials
404	Not Found	The id does not exist, or the URL is not one of ours
500	Internal Server Error	Something went wrong server-side

6. Search Performance: Linear Search vs Dictionary Lookup

The API looks records up by id constantly. Every GET /transactions/{id}, every PUT, every DELETE needs to find one record among 1,691. There are two natural ways to do that, and the difference between them matters once the dataset grows.

We built both and benchmarked them against the full dataset.

The two methods

Linear search walks the list of transactions one at a time, checking each id until it finds the right one. On average it checks half the records. Worst case, all of them.

Dictionary lookup builds a dictionary of id → transaction once, then fetches by key. Python's hash function computes where the value lives in constant time.

The benchmark

We sampled 20 ids spread evenly across the dataset and repeated each measurement 1,000 times.

Position in the list	Linear search (μs)	Dictionary (μs)	Speed -up	Linear comparisons	Dictionary comparisons
1	0.079	0.063	1.3×	1	1
421	10.207	0.065	158.0 ×	421	1
841	20.464	0.062	328.5 ×	841	1
1261	30.529	0.064	476.1 ×	1261	1
1597	39.065	0.063	616.8 ×	1597	1
Average	19.355	0.064	304.5 ×	799.0	1.0

The dictionary won by roughly 304× on average.

Why the dictionary is faster

Linear search is $O(n)$. It starts at the beginning and checks each record in turn, so a record near the end costs far more than one near the start. You can see that in the table: the first record needs 1 comparison, but a record at position 1,597 needs 1,597 of them.

Dictionary lookup is $O(1)$ on average. Python runs the id through a hash function, and the answer tells it exactly where the record sits, so it goes straight there. That is one comparison whether the dictionary holds ten records or ten thousand.

The comparison counts are worth paying attention to because they do not depend on the computer. A faster laptop would shrink both time columns, but linear search would still need 1,597 comparisons and the dictionary would still need 1. The gap is in how the two methods work, not in how fast the machine is.

What else could work

A dictionary is not the only thing that beats a linear scan:

- Binary search on a list sorted by id. That is $O(\log n)$ because each step throws away half of what is left. It needs no extra memory, but the list has to stay sorted.
- A B-tree, or a normal database index. This is what a real database uses for a primary key. It stays fast and it can also answer range questions a dictionary cannot, like "every transaction between these two dates".
- A second dictionary on another field — sender or transaction_type — if you need fast lookups on something other than the id.

Why this shaped the API

The API keeps its transactions in a dictionary keyed by id. That is not just a detail — it is the reason GET /transactions/{id}, PUT /transactions/{id}, and DELETE /transactions/{id} all run in constant time regardless of how large the dataset grows. The benchmark is not academic. It directly justified the storage structure

7. Reflection on Basic Auth Limitations

Basic Authentication is what this API implements, and it is implemented correctly. Credential comparison uses `hmac.compare_digest` to avoid timing attacks, credentials come from

environment variables rather than being hardcoded, and every failure mode returns a proper 401.

But Basic Auth has structural weaknesses that matter, particularly for financial data:

- Base64 is not encryption. Decoding `YWRtaW46bW9tbzlwMjU=` back to `admin:momo2025` takes one shell command. Over plain HTTP this is a real exposure; over HTTPS it is mitigated, but the other problems remain.
- Credentials are resent on every request. Every call re-sends the password. Each transmission is a chance for it to end up in a proxy log or an error report.
- No expiry, no revocation. A Basic Auth password is valid until someone changes it, and changing it is the only way to invalidate it. If several clients share the credential, rotating it breaks all of them at once.
- One shared account. The server log can tell you "admin" did something, but not which admin. And there is no way to give a read-only client read-only access.
- No rate limiting. Nothing slows down a brute-force attempt.

A production deployment would use JWT for first-party clients: the client logs in once and receives a signed token with an expiry, and sends that token instead of the password. That fixes most of the above, because passwords stop crossing the network, tokens expire on their own, and the token can carry the user's id and role for a real audit trail. For third-party integrations, OAuth 2.0 is the standard: it lets you hand a third-party app a scoped token and revoke it later without touching anyone's password.

Either way, HTTPS is required, and passwords should be stored salted and hashed with bcrypt or Argon2 on the server side. A longer write-up is available in `security_notes.md`.

8. Running the Server

The API uses only the Python standard library, so there is nothing to install.

```
# Step 1 — Parse the XML and produce the JSON file
python3 dsa/parse_xml.py
```

```
# Step 2 — Start the server
```

```
python3 api/app.py
```

The server prints the URL it is listening on and the username it is configured with. Press Ctrl-C to stop.

To change the credentials or the port:

```
API_USERNAME=myuser API_PASSWORD=mysecret API_PORT=9000 python3  
api/app.py
```

9. Testing

The repository ships with a shell script that exercises every endpoint plus the authentication failure paths.

```
# In one terminal  
python3 api/app.py
```

```
# In another
```

```
bash screenshots/run_tests.sh
```

The script runs nine cases and prints the status code of each, so you can see at a glance whether the API is behaving. The full transcript is saved to `screenshots/api_test_results.txt`.

10. Further Reading

- `docs/security_notes.md` — a longer discussion of Basic Auth's limitations and how to replace it
- `dsa/dsa_comparison.py` — the benchmark comparing linear search to dictionary lookup
- `screenshots/dsa_results.txt` — the raw benchmark output
- `docs/report.pdf` — the full project report, including the DSA analysis and reflection